

Modelo de Parcial 2 – Aprendizaje por Refuerzo

Inteligencia Artificial y Neurociencias
Universidad Torcuato Di Tella

Formato. Combina **ejercicios prácticos** (seguimiento manual de algoritmos, cálculo de retornos y valores) con **preguntas de interpretación** sobre las fórmulas y conceptos. Debajo de cada consigna se da una *respuesta posible* a modo de guía.

1. Bandidos: actualización incremental (práctico)

Un algoritmo de bandidos con promedio muestral tiene, para la acción a , la estimación $Q_n(a) = 3$ luego de haberla tomado $n - 1 = 4$ veces. Se vuelve a elegir a y se obtiene $R_n = 8$. (a) Calcular $Q_{n+1}(a)$ con la fórmula incremental. (b) ¿Por qué esta forma es preferible a guardar todas las recompensas y promediar?

Respuesta posible. (a) $n = 5$, entonces

$$Q_{n+1}(a) = Q_n + \frac{1}{n} [R_n - Q_n] = 3 + \frac{1}{5} [8 - 3] = 3 + 1 = 4.$$

(b) La actualización incremental requiere almacenar solo Q_n y n (memoria $O(1)$) y cada actualización cuesta $O(1)$, en vez de guardar la lista completa de recompensas y recomputar la media (memoria y tiempo crecientes con la cantidad de muestras).

2. Bandidos (interpretación)

¿Es equivalente usar una política ε -greedy con $\varepsilon = 0,1$ durante 1000 pasos a primero ejecutar 100 pasos puramente exploratorios y luego 900 pasos puramente greedy? Justifique. Explique también la idea de los valores iniciales optimistas como forma de exploración.

Respuesta posible. **No** son equivalentes. Con ε -greedy la exploración está **distribuida** a lo largo de todo el proceso: sigue explorando (aprox. un 10 % del tiempo) incluso al final, lo que le permite corregir estimaciones erróneas y adaptarse (útil si el problema no es estacionario). El esquema “explorar primero, explotar después” fija las estimaciones tras la fase inicial: si por azar una buena acción quedó subestimada en esos primeros pasos, ya nunca la vuelve a probar. Además, durante la fase greedy no obtiene más información de las acciones no elegidas.

Valores iniciales optimistas: se inicializa $Q_1(a)$ con un valor muy alto para toda a . Como toda recompensa real resulta “decepcionante” frente a la estimación inflada, el error $[R - Q]$ es negativo y el agente se ve *empujado* a probar las otras acciones (que siguen con valor alto). Así se logra una exploración inicial sistemática sin necesidad de aleatoriedad. Su limitación: es un mecanismo transitorio, poco útil en problemas no estacionarios.

3. MDP: funciones de valor (práctico)

Ambiente determinístico en línea: $A \rightarrow B \rightarrow C \rightarrow D$, con D terminal. **Recompensas:** $A \rightarrow B$ da 1, $B \rightarrow C$ da 2, $C \rightarrow D$ da 3. Con la política π que avanza siempre a la derecha, calcular $v_\pi(A), v_\pi(B), v_\pi(C)$ para $\gamma = 0$, $\gamma = 0,5$ y $\gamma = 1$.

Respuesta posible. Usando $v_\pi(s) = r + \gamma v_\pi(s')$ desde el final ($v_\pi(D) = 0$):

$$v_\pi(C) = 3 + \gamma \cdot 0 = 3, \quad v_\pi(B) = 2 + \gamma v_\pi(C), \quad v_\pi(A) = 1 + \gamma v_\pi(B).$$

γ	$v_\pi(A)$	$v_\pi(B)$	$v_\pi(C)$
0	1	2	3
0,5	2,75	3,5	3
1	6	5	3

(Con $\gamma = 0,5$: $v_\pi(B) = 2 + 0,5 \cdot 3 = 3,5$; $v_\pi(A) = 1 + 0,5 \cdot 3,5 = 2,75$. Con $\gamma = 1$: $v_\pi(B) = 5$, $v_\pi(A) = 6$.) Como π es determinística, $q_\pi(s, \rightarrow) = v_\pi(s)$ para la acción seguida.

4. Monte Carlo (práctico)

Ambiente con estados $\{s_1, s_2, s_3\}$ (s_3 terminal) **y acciones** $\{a, b\}$. **Se corre Monte Carlo para estimar π_* con $\gamma = 1$ y Q inicializada en 0. Episodios:** **Ep. 1:** $s_1, a, +2, s_2, a, +3, s_3$; **Ep. 2:** $s_1, a, +1, s_2, b, +4, s_3$. **Indicar Retornos(S, A) y $Q(S, A)$ al final de cada episodio, y la acción greedy en cada estado al comenzar el tercer episodio.**

Respuesta posible. Los retornos ($\gamma = 1$) son la suma de recompensas desde cada par (S, A) .

Fin Ep. 1: $G(s_1, a) = 2 + 3 = 5$, $G(s_2, a) = 3$. $\text{Ret}(s_1, a) = [5] \Rightarrow Q(s_1, a) = 5$; $\text{Ret}(s_2, a) = [3] \Rightarrow Q(s_2, a) = 3$.

Fin Ep. 2: $G(s_1, a) = 1 + 4 = 5$, $G(s_2, b) = 4$. $\text{Ret}(s_1, a) = [5, 5] \Rightarrow Q(s_1, a) = 5$; $\text{Ret}(s_2, b) = [4] \Rightarrow Q(s_2, b) = 4$.

Tabla: $Q(s_1, a) = 5$, $Q(s_1, b) = 0$, $Q(s_2, a) = 3$, $Q(s_2, b) = 4$. **Greedy** al comenzar Ep. 3: en s_1 elegir a ($5 > 0$); en s_2 elegir b ($4 > 3$).

5. MC vs TD (interpretación)

¿Cuál es la diferencia fundamental entre los métodos Monte Carlo y los de Diferencias Temporales en cuándo y cómo actualizan Q ? Defina bootstrapping y dé una situación donde TD tenga ventaja práctica sobre MC.

Respuesta posible. **Monte Carlo** espera al final del episodio y actualiza usando el retorno real medido G_t : $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G_t - Q(S_t, A_t)]$. **TD** actualiza tras cada paso, sin esperar el final, usando $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ como objetivo.

Bootstrapping: estimar un valor a partir de otras estimaciones propias (aquí, $Q(S_{t+1}, \cdot)$) en lugar de esperar el resultado real completo. TD hace bootstrapping; MC no.

Ventaja de TD: en tareas continuas (sin fin) o con episodios muy largos, MC no podría actualizar (no hay retorno final que medir), mientras que TD aprende “sobre la marcha”.

Ejemplo: un agente en un videojuego con muchos estados aprende en tiempo real con TD, mientras que MC tendría que completar la partida entera para cada actualización.

6. Deep Q-Learning (interpretación)

¿Por qué los métodos tabulares no sirven en problemas como jugar al Atari desde los píxeles? ¿Qué se hace en su lugar? Explique brevemente para qué sirven el replay de experiencias y la target network en DQN.

Respuesta posible. Los métodos tabulares guardan un valor por cada par (s, a) en una tabla Q , lo cual es imposible cuando el espacio de estados $\mathcal{S}$ es gigantesco: la cantidad de pantallas posibles de un juego es combinatoriamente enorme, y además dos pantallas casi idénticas cuentan como estados distintos (no hay generalización). En su lugar se usa **aproximación de la función de valor**: se reemplaza la tabla por una función parametrizada $\hat{q}(s, a, \mathbf{w})$ (una red neuronal con pesos $\mathbf{w}$, $d \ll |\mathcal{S}|$) que se ajusta con descenso por gradiente (semi-gradiente), permitiendo generalizar a estados nunca vistos.

Replay de experiencias: se almacenan las transiciones $e_t = (S_t, A_t, R_{t+1}, S_{t+1})$ en una memoria y cada actualización se hace sobre un *minibatch* muestreado al azar; esto rompe la correlación temporal entre muestras consecutivas y estabiliza el entrenamiento. **Target network:** una copia de la red que se usa para computar el objetivo y se actualiza solo cada cierta cantidad de iteraciones; al mantener el objetivo “fijo” un rato, evita que la red persiga un blanco que se mueve con cada paso.